

不构造预测分析表，判断下面文法是否LL(1)文法。

$$(1) \quad S \rightarrow Ra \mid a \quad R \rightarrow Sb \mid b$$

$$(2) \quad S \rightarrow aAc \mid b \quad A \rightarrow a \mid b \mid \varepsilon$$



## 4.6 自底向上分析

一般而言，自底向上的分析算法功能比自顶向下的方法强大（例如：左递归的文法），但也更复杂。

- 方法：从句子出发，反复利用产生式做归约（用产生式的左部替代右部），逐步构造语法树，最后得到文法的开始符号
- 核心：寻找句柄

所谓自下而上分析就是自左至右扫描输入串，自下而上进行分析；通过反复查找当前句型的句柄(最左直接短语)，并使用产生式规则将找到的句柄归约为相应的非终结符。这样逐步进行“归约”，直到归到文法的开始符号；或者说，从语法树的末端开始，步步向上“归约”，直到根结点。



自下而上分析法是一种“移进—归约”法，这是因为在自下而上分析过程采用了一个先进后出的分析栈。分析开始后，把输入符号自左至右逐个移进分析栈，并且边移入边分析，一旦栈顶的符号串形成某个句型的句柄时就进行一次归约，即用相应产生式的左部非终结符替换当前句柄。接下来继续查看栈顶是否形成新的句柄，若为句柄则再进行归约；若栈顶不是句柄则继续向栈中移进后续输入符号。不断重复这一过程，直到将整个输入串处理完毕。若此时分析栈只剩有文法的开始符号则分析成功，即确认输入串是文法的一个句子；否则，即认为分析失败。

我们举一个简单的例子来说明这种分析过程。假设一文法 $G[S]$ 为：

$$G[S]: S \rightarrow aAbB$$

$$A \rightarrow c \mid Ac$$

$$B \rightarrow d$$

试对输入串 $accbd$ 进行分析，检查该符号串是否是 $G[S]$ 的一个句子。

我们仍将“#”作为输入串的界符(括号)，并将输入串前的“#”放入分析栈，接着将输入串的符号依次进栈，具体分析过程如表3.6所示。

表3.6 对输入串accbd自下而上的分析过程

| 步骤 | 分析栈   | 句 柄  | 输入串    | 动 作                        |
|----|-------|------|--------|----------------------------|
| 1  | #     |      | accbd# | 移进                         |
| 2  | #a    |      | ccbd#  | 移进                         |
| 3  | #ac   |      | cbd#   | 移进                         |
| 4  | #aA   | c    | cbd#   | 归约( $A \rightarrow c$ )    |
| 5  | #aAc  |      | bd#    | 移进                         |
| 6  | #aA   | Ac   | bd#    | 归约( $A \rightarrow Ac$ )   |
| 7  | #aAb  |      | d#     | 移进                         |
| 8  | #aAbd |      | #      | 移进                         |
| 9  | #aAbB | d    | #      | 归约( $B \rightarrow d$ )    |
| 10 | #S    | aAbB | #      | 归约( $S \rightarrow aAbB$ ) |

上述语法分析过程可以用一棵分析树来表示。在自下而上分析过程中，每一步归约都可以画出一棵子树来，随着归约的完成，这些子树被连成一棵完整的分析树。根据表3.6构造分析树的过程如图3-15所示。

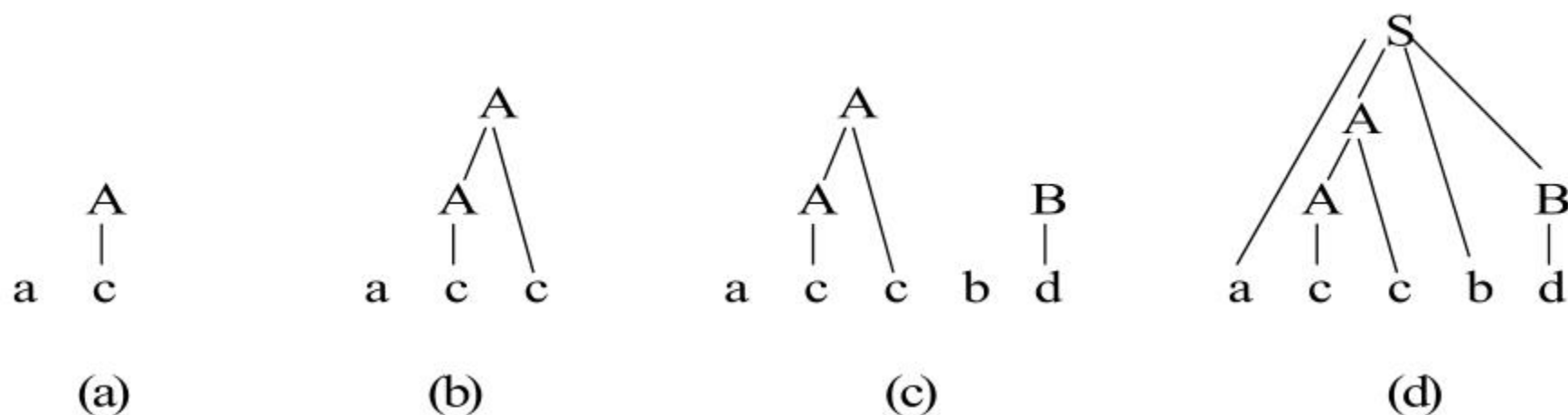


图3-15 自下而上构造分析树的过程

例:

$S \rightarrow a A c B e$

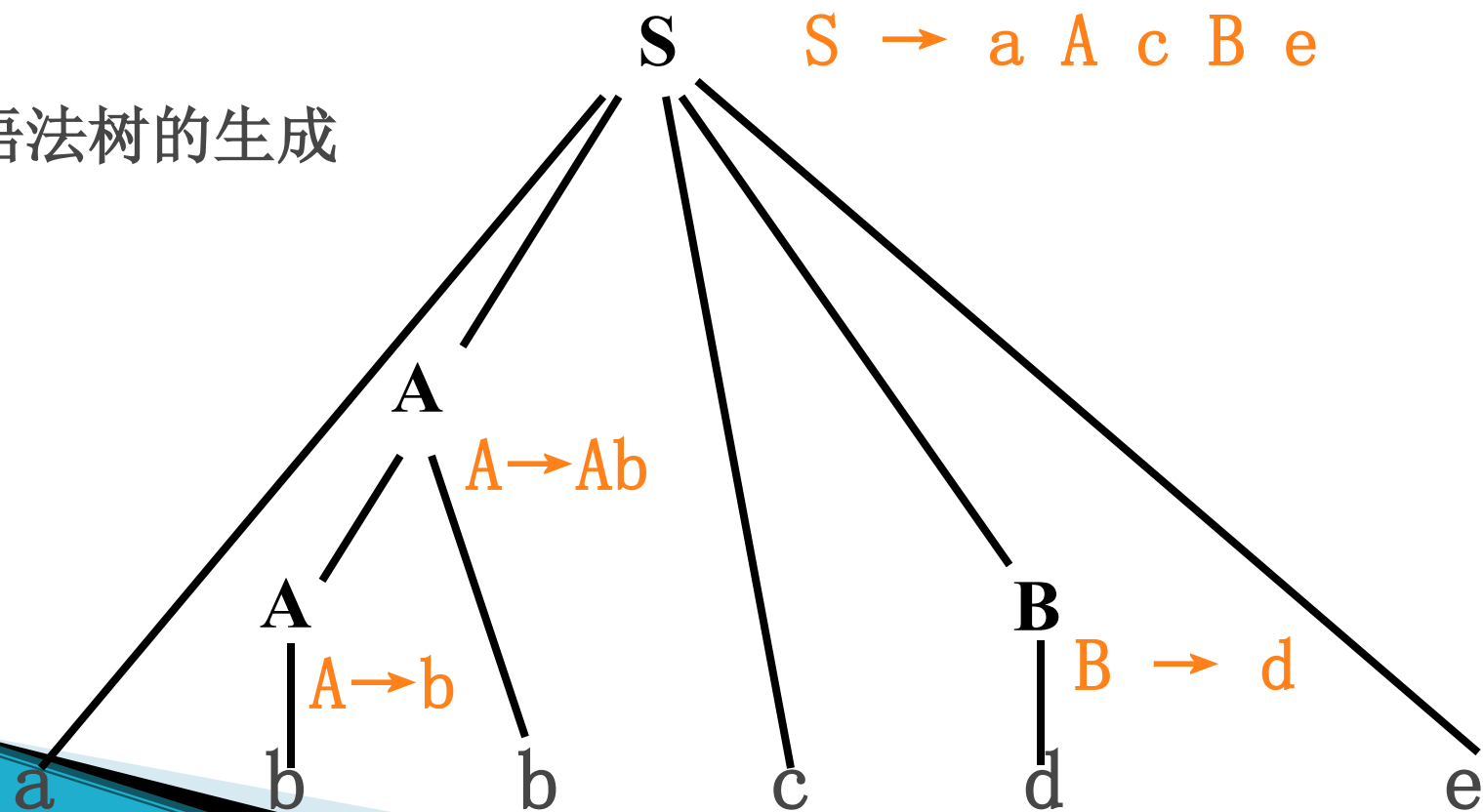
$A \rightarrow A b \mid b$

$B \rightarrow d$

句子abbcede的分析过程:

$a\underline{b}cde \rightarrow a\underline{A}bcde \rightarrow a\underline{A}c\underline{d}e$   
 $\rightarrow a\underline{A}c\underline{B}e \rightarrow \underline{S}$

语法树的生成



## 4.6.1、句柄：最左直接短语

➤ 设文法  $G = \{ V_T, V_N, P, S \}$

若  $S \xRightarrow{*} \alpha A \delta$  且  $A \xRightarrow{+} \beta$ ，其中  $\alpha, \beta, \delta \in (V_N \cup V_T)^*$

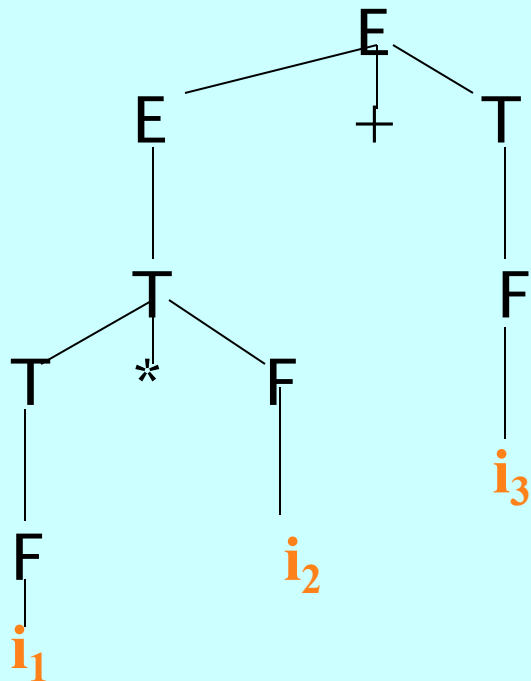
则称  $\beta$  是句型  $\alpha \beta \delta$  相对于非终结符  $A$  的**短语**。（即**短语是句型的一部分**，能由某个非终结符经1或多步推导得来。

➤ 若  $S \xRightarrow{*} \alpha A \delta$  且  $A \Rightarrow \beta$ ，则称  $\beta$  是句型  $\alpha \beta \delta$  的**直接短语**（**简单短语**）。（是某个产生式的右部）

➤ 一个句型的最左直接短语称为该句型的**句柄**。

短语的两个条件缺一不可。仅有  $A \xRightarrow{+} \beta$  未必意味着  $\beta$  就是句型  $\alpha \beta \delta$  的一个短语，还需要有  $S \xRightarrow{*} \alpha A \delta$  这一条件加以限制；也即短语属于该句型的组成部分，不能破坏这种句型结构的限制。

## 短语、直接短语和句柄：



$G[E]: E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid i$

句型:  $i_1*i_2+i_3$

短语:  $i_1*i_2+i_3, i_1*i_2, i_1, i_2, i_3$

直接短语:  $i_1, i_2, i_3$

句柄:  $i_1$

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow T*F+T \Rightarrow F*F+T \Rightarrow i_1*F+T \Rightarrow i_1*i_2+T \Rightarrow i_1*i_2+F$   
 $\Rightarrow i_1*i_2+i_3$  ( $i_3$ 是句型 $i_1*i_2+i_3$ 相对于 $F$ 或相对于 $T$ 的短语)

$E \Rightarrow E+T \Rightarrow E+F \Rightarrow E+i_3 \Rightarrow T*F+i_3 \Rightarrow T*i_2+i_3 \Rightarrow F*i_2+i_3 \Rightarrow i_1*i_2+i_3$

( $i_1$ 是 $i_1*i_2+i_3$ 相对于 $F$ 或相对于 $T$ 的短语,  $i_3$ 是 $E+i_3$ 相对于 $T$ 或 $F$ 的短语)

( $T*F$ 是 $T*F+i_3$ 的最左直接短语)

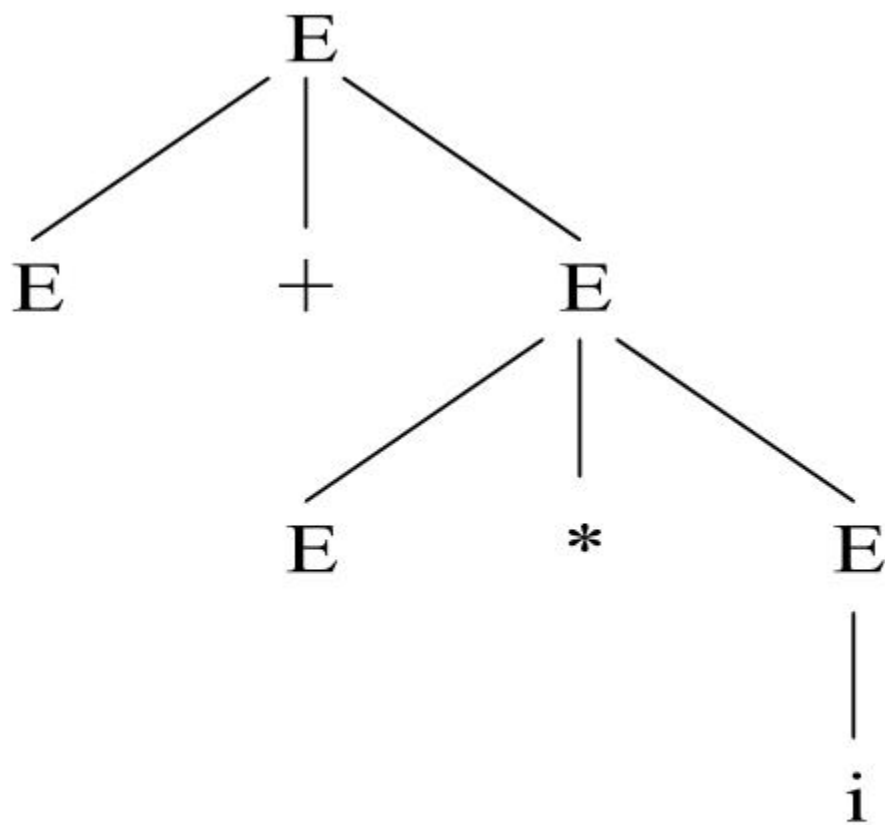


图3-5  $E+E*i$ 的语法树

对图3-5所示的关于句型 $E+E*i$ 的语法树来说，它有3棵子树，即3个短语，分别为 $i$ 、 $E*i$ 和 $E+E*i$ ；而直接短语、句柄和素短语均为 $i$ 。

### 3. 句柄

一个句型的最左直接短语称为该句型的句柄。注意，一个句型的直接短语可能不只一个，但最左直接短语则是惟一的。如对 $S \xRightarrow{*} \alpha A \delta \Rightarrow \alpha \beta \delta$ 来说，将句型 $\alpha \beta \delta$ 中的句柄 $\beta$ 用产生式的左部符号代替便得到新句型 $\alpha A \delta$ ，这是一次规范归约，恰好与规范推导相反。

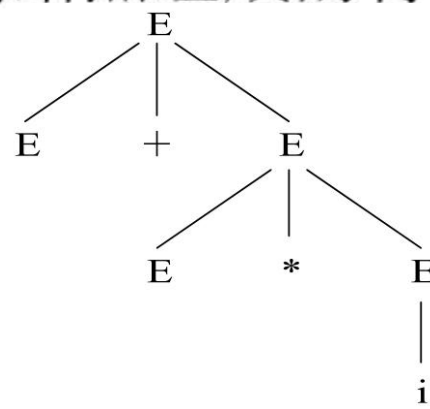
## 2. 子树和短语

语法树的某个结点连同它的所有后代组成了一棵子树，只含有单层分枝的子树称为简单子树。

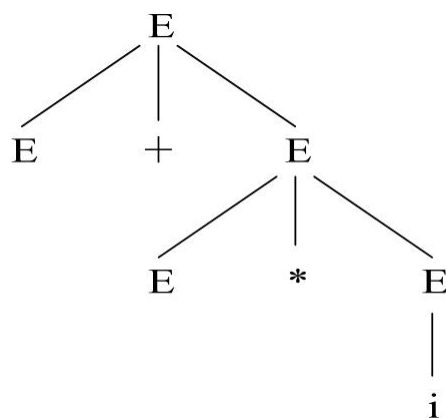
子树与短语的关系十分密切，根据子树的概念，句型的短语、直接短语、句柄和素短语的直观解释如下：

(1) 短语：子树的末端结点(即树叶)组成的符号串是相对于子树根的短语；

(2) 直接短语：简单子树的末端结点组成的符号串是相对于简单子树根的直接短语；



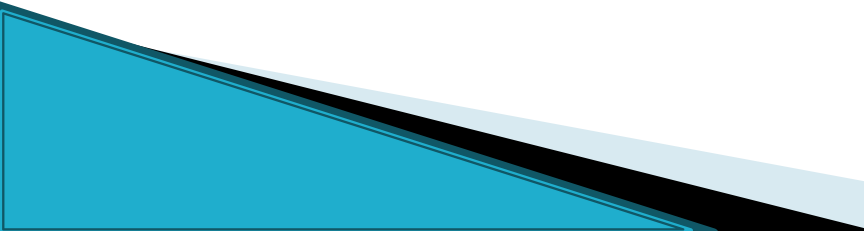
(3) 句柄：最左简单子树的末端结点组成的符号串为句柄；



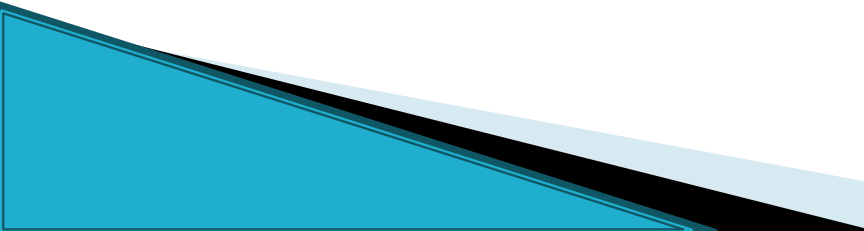
从建立分析树的过程可以清楚地看出，自下而上分析过程的每一步归约确实都是归约当前句型的句柄；也就是说句柄一旦形成，则它总是出现在分析栈的栈顶而不会出现在栈的中间。由于每一步归约都是把栈顶的一串符号用某个产生式的左部符号替换，因而可以把栈顶的这样一串符号称为“可归约串”。初看起来，“移进—归约”似乎很简单，其实不然。在上例分析进行到第6步时，如果我们不是选择规则 $A \rightarrow Ac$ 而是选择规则 $A \rightarrow c$ 进行归约，也即把 $c$ 看作句柄的话，则最终就无法达到归约到 $S$ 的目的，从而也就无法得知输入串 $accbd$ 是一个符合文法的句子。

为什么知道此时栈顶的 $Ac$ 形成“可归约串”而 $c$ 不是“可归约串”呢？这就需要精确定义“可归约串”这个直观概念，这也是自下而上分析的关键问题。

我们在3.2.1节知道了推导与短语的概念，并且知道归约是推导的逆过程。这里，我们进一步指出：如果文法 $G[S]$ 是无二义的，那么规范推导(最右推导)的逆过程必是规范归约(最左归约)。



请注意句柄的“最左”特征，这一点对于“移进一归约”来说很重要，因为句柄的“最左”性和分析栈的栈顶两者是相关的。对于规范句型(规范推导所得的句型)来说，句柄的后面不会出现非终结符(也即句柄的后面只能出现终结符)。基于这一点，我们可用句柄来刻画“移进一归约”过程的“可归约串”。因此，规范归约的实质是，在移进过程中，当发现栈顶呈现句柄时就用相应产生式的左部符号进行替换(即归约)。



上述分析有助于理解为什么规范归约所得的分析树恰好就是语法树。

为了加深对“句柄”和“归约”这些重要概念的理解，我们使用修剪语法树的办法来进一步阐明自下而上的分析过程。

一棵语法树的一个子树是由该树的某个结点(作为子树的根)连同它的所有子孙(如果有的话)组成的，也即一个子树的所有树叶自左至右排列起来形成一个相对于子树根的短语，并且一个句型的句柄是这个句型所对应的语法树中最左那个子树树叶自左至右的排列；这个子树只有(而且必须有)父子两代，没有第三代。

例如，对图3-15(d)所示的语法树，我们采用修剪语法树的方法来实现归约，也即每次寻找当前语法树的句柄(在语法树中用虚线勾出)，然后将句柄中的树叶剪去(即实现一次归约)；这样不断地修剪下去，当剪到只剩下根结时，就完成了整个归约过程，如图3-16所示。

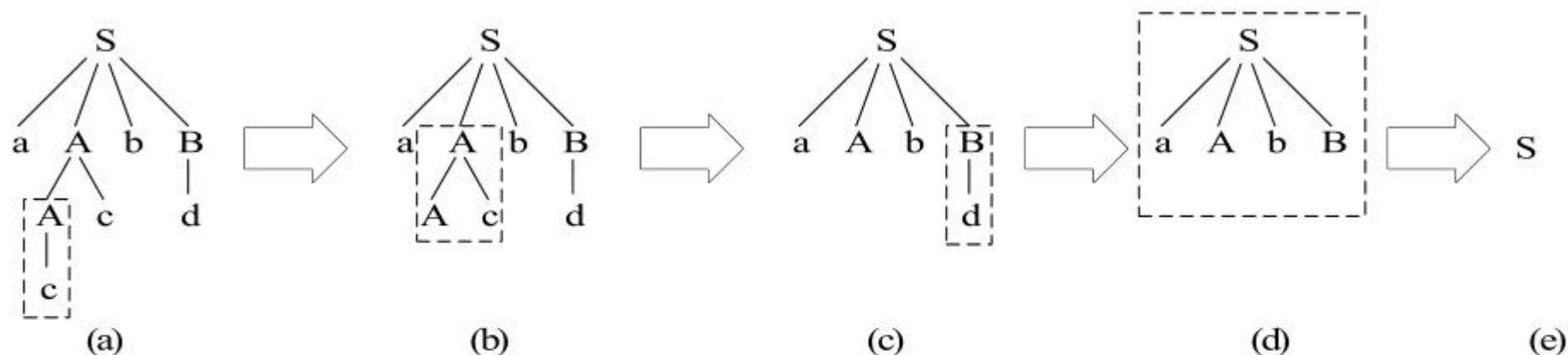


图3-16 修剪语法树实现归约

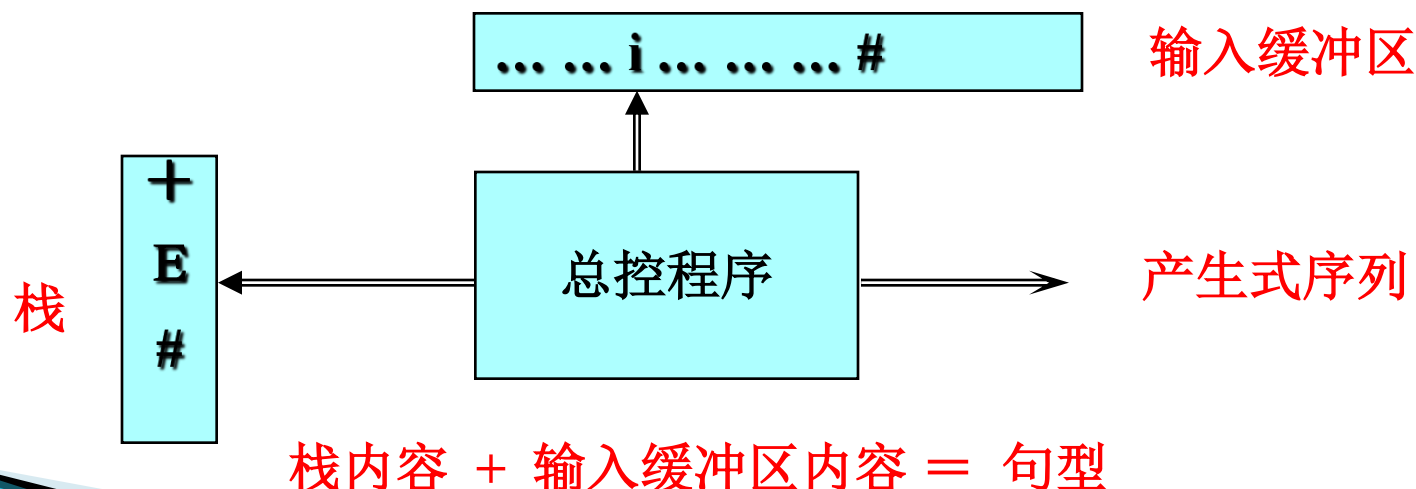
## 4.6.2、移进归约分析器 (Shift-Reduce)

结构

- **栈**: 保存语法符号
- **输入缓冲区**: 保存输入符号串
- **总控程序**: 控制分析过程, 输出产生式序列

状态:

- **初始**: 栈上只有 #, 输入缓冲区有 W#
- **正常结束**: 栈上为 #S, 输入缓冲区只有 #



# 分析器的四种动作

- 1) 移进：将下一输入符号移入栈
- 2) 归约：用产生式左侧的非终结符替换栈顶的句柄
- 3) 接受：分析成功
- 4) 出错：出错处理

自底向上分析方法是一种移进—归约过程，当分析栈顶符号串形成句柄时就采取归约动作。因而自底向上分析法的关键问题是分析过程中**如何确定句柄**。各种自底向上分析方法**确定句柄的方法不同**

例：用分析器对表达式  $i+i*i$  进行分析

- $E \rightarrow i$
- $E \rightarrow E * E$
- $E \rightarrow E + E$
- $E \rightarrow (E)$

## 例：表达式 $i+i*i$ 的分析

| 动作                         | 栈           | 输入缓冲区           |
|----------------------------|-------------|-----------------|
| 1)                         | #           | $i_1+i_2*i_3\#$ |
| 2) 移进                      | $\#i_1$     | $+i_2*i_3\#$    |
| 3) 归约 $E \rightarrow i$    | $\#E$       | $+i_2*i_3\#$    |
| 4) 移进                      | $\#E+$      | $i_2*i_3\#$     |
| 5) 移进                      | $\#E+i_2$   | $*i_3\#$        |
| 6) 归约 $E \rightarrow i$    | $\#E+E$     | $*i_3\#$        |
| 7) 移进                      | $\#E+E*$    | $i_3\#$         |
| 8) 移进                      | $\#E+E*i_3$ | $\#$            |
| 9) 归约 $E \rightarrow i$    | $\#E+E*E$   | $\#$            |
| 10) 归约 $E \rightarrow E*E$ | $\#E+E$     | $\#$            |
| 11) 归约 $E \rightarrow E+E$ | $\#E$       | $\#$            |
| 12) 接受                     |             |                 |

# 移进归约分析中的冲突

文法复杂时出现:

1) **移进归约冲突** ( $C_1 \rightarrow A \beta \alpha$ ,  $C_2 \rightarrow A \beta$ )

✓ 上例中的 6) 可以移进  $i$  或 归约  $E \rightarrow E+E$

✓ 例:  $S \rightarrow \text{if } E \text{ then } S$

$\quad \quad \quad | \text{if } E \text{ then } S \text{ else } S$

如输入  $\text{if } E \text{ then if } E \text{ then } S \text{ else } S$

分析某一时刻, 栈的内容:  $\text{if } E \text{ then if } E \text{ then } S$

此时是归约  $\text{if } E \text{ then } S$  还是移进  $\text{else}$  ?

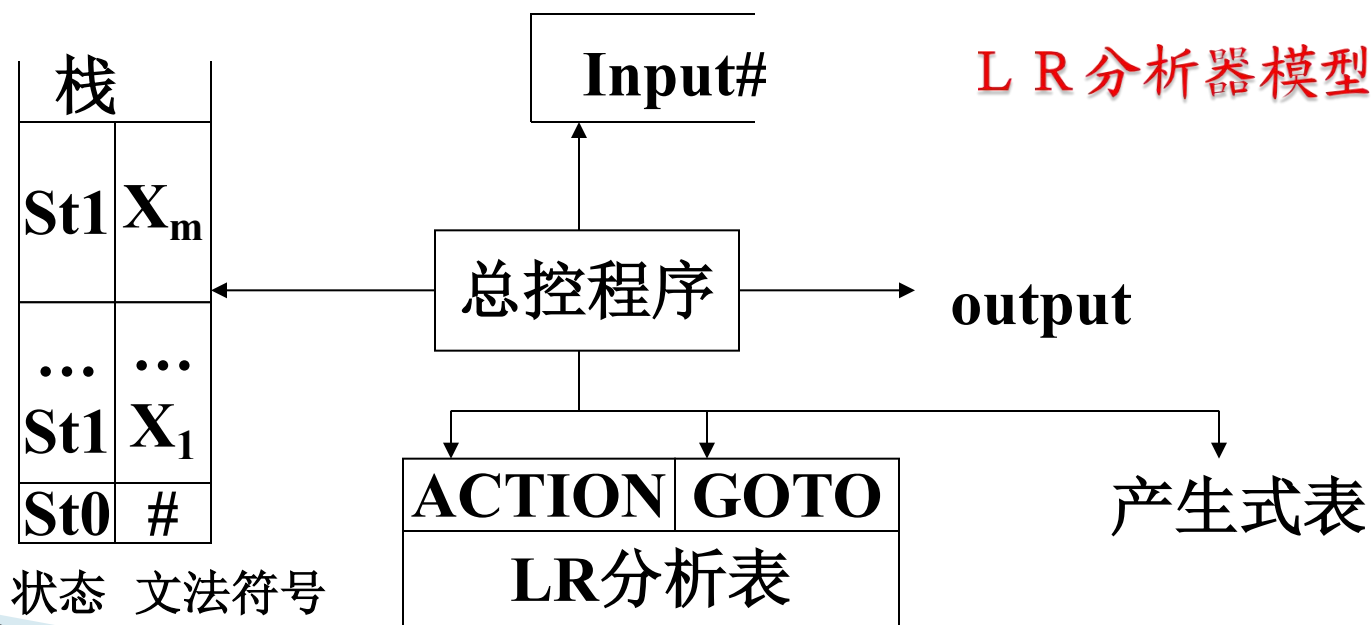
2) **归约归约冲突** ( $C_1 \rightarrow \alpha A \beta$ ,  $C_2 \rightarrow A \beta$ )

存在两个可用的产生式

# 一种特定的移进-归约实现技术：L R (K) 分析

(L：从左到右扫描输入串；R：最右推导)

根据**当前分析栈中的符号串**和**向右顺序查看输入串的K个**  
**( $K \geq 0$ ) 符号**唯一地确定是移进还是归约以及用哪个产生式归约，因而也就能唯一地确定句柄。包括LR(0)、SLR(1)、LR(1)等方法



## 3.5 LR分析法

LR分析法是一种自下而上进行规范归约的语法分析方法，LR指“自左向右扫描和自下而上进行归约”。LR分析法比递归下降分析法、LL(1)分析法和算符优先分析法对文法的限制要少得多，对大多数用无二义的上下文无关文法描述的语言都可以用LR分析器予以识别，而且速度快，并能准确、及时地指出输入串的任何语法错误及出错位置。LR分析法的一个主要缺点是，若用手工构造分析器则工作量相当大，因此必须求助于自动产生LR分析器的产生器。

**LR分析是特定的一种shift-reduce实现技术**

### 3.5.1 LR分析器的工作原理

我们知道，规范归约(最左归约，即最右推导的逆过程)的关键问题是寻找句柄。LR分析法的基本思想是：在规范归约过程中，一方面记住已移进和归约出的整个符号串，即记住“历史”；另一方面根据所用的产生式推测未来可能遇到的输入符号，即对未来进行“展望”。当一串貌似句柄的符号串呈现于分析栈的顶端时，我们希望能够根据所记载的“历史”和“展望”以及“现实”的输入符号等三方面的材料，来确定栈顶的符号是否构成相对某一产生式的句柄。

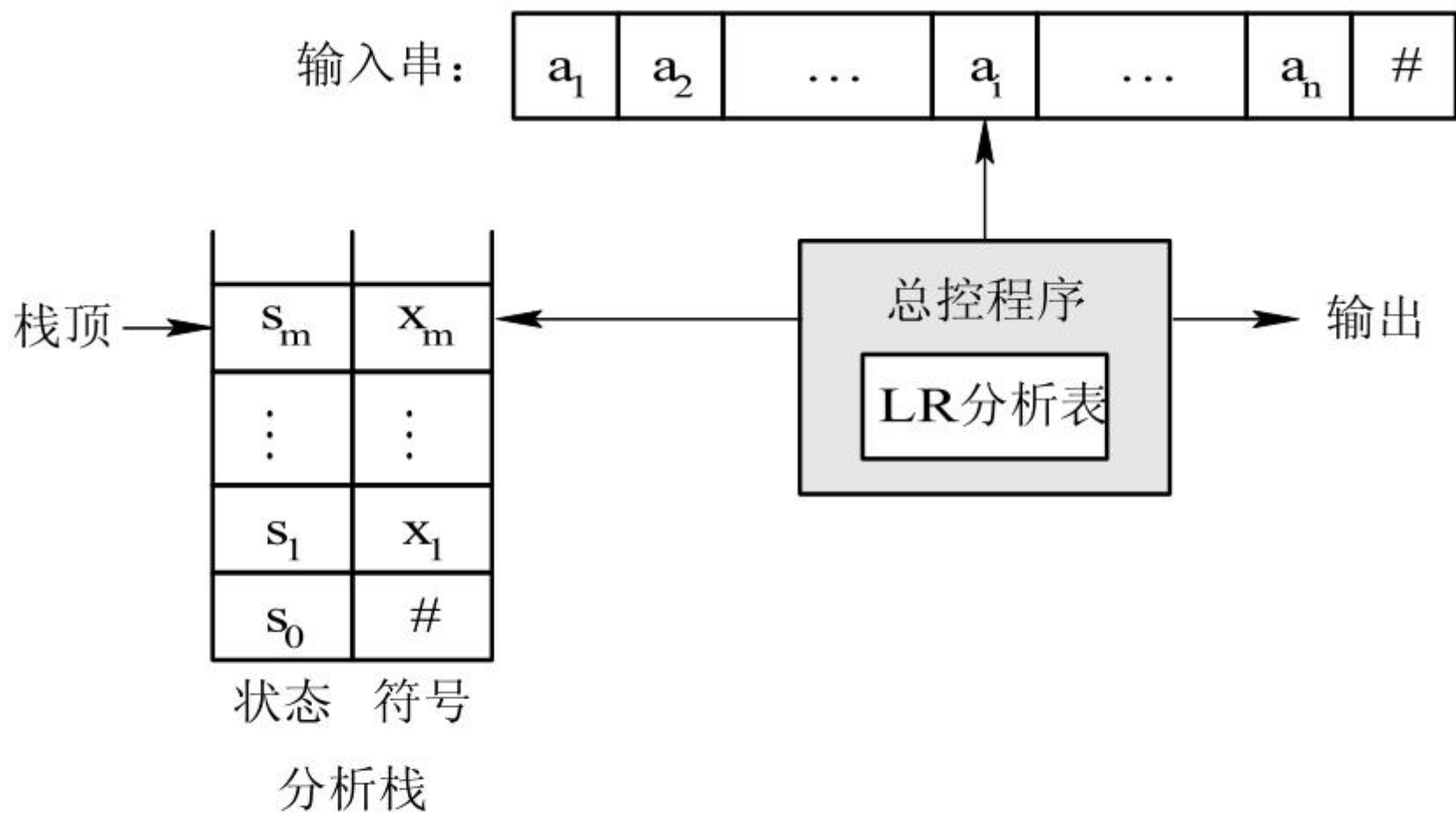


图3-21 LR分析器结构示意

一个LR分析器实质上是一个带先进后出存储器(栈)的确定有限状态自动机。我们将把“历史”和“展望”材料综合抽象成某些“状态”，而分析栈(先进后出存储器)则用来存放这些状态；栈里的每个状态概括了从分析开始直到某一归约阶段的全部“历史”和“展望”资料。任何时候，栈顶的状态都代表了整个“历史”和已推测出的“展望”。LR分析器的每一步工作都是由栈顶状态和现行输入符号所惟一决定的。为了有助于明确归约手续，我们把已归约出的文法符号串也同时放在栈中(实际上可不必进栈)。栈的每一项内容包括状态s和文法符号X两部分。 $(s_0, \#)$ 为分析开始前预先放入栈里的初始状态和句子括号；栈顶状态为 $s_m$ ，符号串 $X_1X_2\dots X_m$ 是至今已移进归约出的文法符号串。

LR分析表是LR分析器的核心部分。一张LR分析表包括两部分：一部分是“动作”(ACTION)表，另一部分是“状态转换”(GOTO)表；它们都是二维数组。ACTION[s,a]规定了当状态s面临输入符号a时应采取什么动作，而GOTO[s, X]规定了状态s面对文法符号X(终结符或非终结符)时的下一状态是什么。显然，GOTO[s, X]定义了一个以文法符号为字母表的DFA。

每一项ACTION[s, a]所规定的动作是以下四种情况之一：

(1) 移进：使(s, a)的下一状态 $s' = \text{ACTION}[s, a]$ 和输入符号a进栈(注：对终结符a来说，

下一状态 $s' = \text{GOTO}[s, a]$ 的值实际上是放在 $\text{ACTION}[s, a]$ 中的), 下一输入符号变成现行输入符号。

(2) 归约: 指用某一产生式 $A \rightarrow \beta$ 进行归约。假若 $\beta$ 的长度为 $\gamma$ , 则归约的动作是去掉栈顶的 $\gamma$ 个项, 即使状态 $s_{m-\gamma}$ 变成栈顶状态, 然后使 $(s_{m-\gamma}, A)$ 的下一状态 $s' = \text{GOTO}[s_{m-\gamma}, A]$ 和文法符号(非终结符) $A$ 进栈。归约的动作不改变现行输入符号, 执行归约的动作意味着呈现于栈顶的符号串 $X_{m-\gamma+1} \dots X_m$ 是一个相对于 $A$ 的句柄。

(3) 接受: 宣布分析成功, 停止分析器的工作。

(4) 报错: 报告发现源程序含有错误, 调用出错处理程序。

LR分析器的总控程序本身的工作十分简单, 它的任何一步只需按分析栈的栈顶状态 $s$ 和现行输入符号 $a$ 执行 $ACTION[s,a]$ 所规定的动作即可。

shift

| ACTION |           |    |           |    |    |     | GOTO |   |   |
|--------|-----------|----|-----------|----|----|-----|------|---|---|
|        | a         | c  | e         | b  | d  | #   | S    | A | B |
| 0      | <u>S2</u> |    |           |    |    |     | 1    |   |   |
| 1      |           |    |           |    |    | acc |      |   |   |
| 2      |           |    |           | S4 |    |     |      | 3 |   |
| 3      |           | S5 |           | S6 |    |     |      |   |   |
| 4      | r2        | r2 | r2        | r2 | r2 | r2  |      |   |   |
| 5      |           |    |           |    | S8 |     |      |   | 7 |
| 6      | r3        | r3 | r3        | r3 | r3 | r3  |      |   |   |
| 7      |           |    | S9        |    |    |     |      |   |   |
| 8      | r4        | r4 | r4        | r4 | r4 | r4  |      |   |   |
| 9      | <u>r1</u> | r1 | <u>r1</u> | r1 | r1 | r1  |      |   |   |

reduce

statement

例如，表达式文法 $G[E]$ 如下，它对应的LR分析表见表3.13，则语句 $i+i*i$ 的LR分析过程如表3.14所示：

$G[E]: (1) E \rightarrow E+T$

$(2) E \rightarrow T$

$(3) T \rightarrow T * F$

$(4) T \rightarrow F$

$(5) F \rightarrow (E)$

$(6) F \rightarrow i$

其中， $s_j$ 指把下一状态 $j$ 和现行输入符号 $a$ 移进栈； $r_j$ 指按文法的第 $j$ 个产生式进行归约；acc表示分析成功；空白格为出错。

表3.13 LR分析表

| 状态 | ACTION         |                |                |                |                 |                | GOTO |   |    |
|----|----------------|----------------|----------------|----------------|-----------------|----------------|------|---|----|
|    | i              | +              | *              | (              | )               | #              | E    | T | F  |
| 0  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                | 1    | 2 | 3  |
| 1  |                | s <sub>6</sub> |                |                |                 | acc            |      |   |    |
| 2  |                | r <sub>2</sub> | s <sub>7</sub> |                | r <sub>2</sub>  | r <sub>2</sub> |      |   |    |
| 3  |                | r <sub>4</sub> | r <sub>4</sub> |                | r <sub>4</sub>  | r <sub>4</sub> |      |   |    |
| 4  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                | 8    | 2 | 3  |
| 5  |                | r <sub>6</sub> | r <sub>6</sub> |                | r <sub>6</sub>  | r <sub>6</sub> |      |   |    |
| 6  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                |      | 9 | 3  |
| 7  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                |      |   | 10 |
| 8  |                | s <sub>6</sub> |                |                | s <sub>11</sub> |                |      |   |    |
| 9  |                | r <sub>1</sub> | s <sub>7</sub> |                | r <sub>1</sub>  | r <sub>1</sub> |      |   |    |
| 10 |                | r <sub>3</sub> | r <sub>3</sub> |                | r <sub>3</sub>  | r <sub>3</sub> |      |   |    |
| 11 |                | r <sub>5</sub> | r <sub>5</sub> |                | r <sub>5</sub>  | r <sub>5</sub> |      |   |    |

表3.14 i+i\*i的LR分析过程

| 步骤 | 状态栈   | 符号栈  | 输入串    | 动作说明                                  |
|----|-------|------|--------|---------------------------------------|
| 1  | 0     | #    | i+i*i# | ACTION[0,i]=s <sub>5</sub> , 即状态5入栈   |
| 2  | 0 5   | # i  | +i*i#  | r <sub>6</sub> : 用F→i归约且GOTO(0,F)=3入栈 |
| 3  | 0 3   | # F  | +i*i#  | r <sub>4</sub> : 用T→F归约且GOTO(0,T)=2入栈 |
| 4  | 0 2   | # T  | +i*i#  | r <sub>2</sub> : 用E→T归约且GOTO(0,E)=1入栈 |
| 5  | 0 1   | # E  | +i*i#  | ACTION[1,+]=s <sub>6</sub> , 即状态6入栈   |
| 6  | 0 1 6 | # E+ | i*i#   | ACTION[6,i]=s <sub>5</sub> , 即状态5入栈   |

表3.13 LR分析表

| 状态 | ACTION         |                |                |                |                 |                | GOTO |   |    |
|----|----------------|----------------|----------------|----------------|-----------------|----------------|------|---|----|
|    | i              | +              | *              | (              | )               | #              | E    | T | F  |
| 0  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                | 1    | 2 | 3  |
| 1  |                | s <sub>6</sub> |                |                |                 | acc            |      |   |    |
| 2  |                | r <sub>2</sub> | s <sub>7</sub> |                | r <sub>2</sub>  | r <sub>2</sub> |      |   |    |
| 3  |                | r <sub>4</sub> | r <sub>4</sub> |                | r <sub>4</sub>  | r <sub>4</sub> |      |   |    |
| 4  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                | 8    | 2 | 3  |
| 5  |                | r <sub>6</sub> | r <sub>6</sub> |                | r <sub>6</sub>  | r <sub>6</sub> |      |   |    |
| 6  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                |      | 9 | 3  |
| 7  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                |      |   | 10 |
| 8  |                | s <sub>6</sub> |                |                | s <sub>11</sub> |                |      |   |    |
| 9  |                | r <sub>1</sub> | s <sub>7</sub> |                | r <sub>1</sub>  | r <sub>1</sub> |      |   |    |
| 10 |                | r <sub>3</sub> | r <sub>3</sub> |                | r <sub>3</sub>  | r <sub>3</sub> |      |   |    |
| 11 |                | r <sub>5</sub> | r <sub>5</sub> |                | r <sub>5</sub>  | r <sub>5</sub> |      |   |    |

|    |              |         |     |                                         |
|----|--------------|---------|-----|-----------------------------------------|
| 7  | 0 1 6 5      | # E+i   | *i# | r <sub>6</sub> : 用F→i归约且GOTO(6,F)=3入栈   |
| 8  | 0 1 6 3      | # E+F   | *i# | r <sub>4</sub> : 用T→F归约且GOTO(6,T)=9入栈   |
| 9  | 0 1 6 9      | # E+T   | *i# | ACTION[9,*]=s <sub>7</sub> , 即状态7入栈     |
| 10 | 0 1 6 9 7    | # E+T*  | i#  | ACTION[7,i]=s <sub>5</sub> , 即状态5入栈     |
| 11 | 0 1 6 9 7 5  | # E+T*i | #   | r <sub>6</sub> : 用F→i归约且GOTO(7,F)=10入栈  |
| 12 | 0 1 6 9 7 10 | # E+T*F | #   | r <sub>3</sub> : 用T→T*F归约且GOTO(6,T)=9入栈 |
| 13 | 0 1 6 9      | # E+T   | #   | r <sub>1</sub> : 用E→E+T归约且GOTO(0,E)=1入栈 |
| 14 | 0 1          | # E     | #   | Acc: 分析成功                               |

| 状态 | ACTION         |                |                |                |                 |                | GOTO |   |    |
|----|----------------|----------------|----------------|----------------|-----------------|----------------|------|---|----|
|    | i              | +              | *              | (              | )               | #              | E    | T | F  |
| 0  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                | 1    | 2 | 3  |
| 1  |                | s <sub>6</sub> |                |                |                 | acc            |      |   |    |
| 2  |                | r <sub>2</sub> | s <sub>7</sub> |                | r <sub>2</sub>  | r <sub>2</sub> |      |   |    |
| 3  |                | r <sub>4</sub> | r <sub>4</sub> |                | r <sub>4</sub>  | r <sub>4</sub> |      |   |    |
| 4  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                | 8    | 2 | 3  |
| 5  |                | r <sub>6</sub> | r <sub>6</sub> |                | r <sub>6</sub>  | r <sub>6</sub> |      |   |    |
| 6  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                |      | 9 | 3  |
| 7  | s <sub>5</sub> |                |                | s <sub>4</sub> |                 |                |      |   | 10 |
| 8  |                | s <sub>6</sub> |                |                | s <sub>11</sub> |                |      |   |    |
| 9  |                | r <sub>1</sub> | s <sub>7</sub> |                | r <sub>1</sub>  | r <sub>1</sub> |      |   |    |
| 10 |                | r <sub>3</sub> | r <sub>3</sub> |                | r <sub>3</sub>  | r <sub>3</sub> |      |   |    |
| 11 |                | r <sub>5</sub> | r <sub>5</sub> |                | r <sub>5</sub>  | r <sub>5</sub> |      |   |    |

我们主要关心的问题是如何由文法构造LR分析表。对于一个文法，如果能够构造一张分析表，使得它的每个入口均是惟一确定的，则称这个文法为LR文法。对于一个LR文法，当分析器对输入串进行自左至右扫描时，一旦句柄呈现于栈顶，就能及时对它实行归约。

在有些情况下，LR分析器需要“展望”和实际检查未来的k个输入符号才能决定应采取什么样的“移进—归约”决策。一般而言，一个文法如果能用一个每步最多向前检查k个输入符号的LR分析器进行分析，则这个文法就称为LR(k)文法。

对于一个文法，如果它的任何“移进—归约”分析器都存在这样的情况：尽管栈的内容和下一个输入符号都已了解，但仍无法确定是“移进”还是“归约”，或者无法从几种可能的归约中确定其一，则该文法是非LR(1)的。注意，LR文法肯定是无二义的，一个二义文法决不会是LR文法；但是，LR分析技术可以进行适当修改以适用于分析一定的二义文法。



# LR分析中的一些主要概念

**最右推导：**在推导的任何一步  $\alpha \Rightarrow \beta$ ，其中  $\alpha$ 、 $\beta$  是句型，都是对  $\alpha$  中的最右非终结符进行替换

最右推导被称为**规范推导**。

由规范推导所得的句型称为**规范句型**。

2)  $E \Rightarrow E+T \Rightarrow E+T * F \Rightarrow E+T * i \Rightarrow E+F * i$   
 $\Rightarrow E+i * i \Rightarrow T+i * i \Rightarrow F+i * i \Rightarrow i+i * i$  ( $i+E$  不是规范句型)

**规范归约**是规范推导(最右推导)的逆过程

设  $\alpha$  为文法  $G$  的句子，称序列  $\alpha_n, \dots, \alpha_1, \alpha_0$  为  $\alpha$  的规范归约，其中：

1)  $\alpha_n = \alpha$

2)  $\alpha_0 = S$  (开始符号)

3) 对每个  $i (1 \leq i \leq n)$ ， $\alpha_{i-1}$  是将  $\alpha_i$  中的**句柄**归约后得到的

$E \leq E+T \leq E+T^*F \leq E+T^*i \leq E+T^*i \leq E+T^*i \leq E+F^*i$   
 $\leq E+i^*i \leq E+i^*i \leq E+i^*i \leq T+i^*i \leq F+i^*i \leq i+i^*i$

**规范前缀：**若有规范句型 $\alpha\eta$ ，且 $\eta$ 是终结字符串或空串，则称 $\alpha$ 为规范前缀。（如 $E$ ， $E+$ ， $E+i$ ， $E+i^*$ ）

**规范活前缀：**若规范前缀 $\alpha$ 不含句柄，或含一个句柄且具有形式 $\alpha = \alpha' \pi$ （ $\pi$ 是句柄），则称规范前缀 $\alpha$ 为规范活前缀（如 $E$ ， $E+$ ， $E+i$ ），或称活前缀。

**归约规范活前缀：**若活前缀 $\alpha$ 是含句柄的活前缀，即有 $\alpha = \alpha' \pi$ （ $\pi$ 是句柄），则称活前缀 $\alpha$ 为归约规范活前缀（如 $E+i$ ），或称归约活前缀。

**我们在归约分析的过程中，期待在栈中出现归约活前缀**

## 主要问题:

- 如何判别活前缀: 构造一个识别活前缀的 DFA。
- 如何判别活前缀是归约活前缀
- 按什么产生式进行归约

## 四种技术:

- LR(0)      SLR(1)      LR(1)      LALR(1)
- 共同点: 逻辑结构、工作原理 (状态转移)
- 不同点: 构造分析表的方法、分析能力、实现效率

## LR分析的特征:

- 规范的
  - 符号栈中的符号串必须是活前缀
  - 分析决策依据——栈顶状态和当前输入符号
- 